Une phrase, un timer, une interruption

115200 8N1 • Un message toutes les 2 secondes • Une boucle infinie vide

Les trois idées à relier

La fonction **uart_printstr** parcourt une chaîne. L’**UART** transmet ses caractères. Un **timer** mesure le temps et déclenche une **interruption** pour demander l’affichage périodique.

CHAÎNE

Une suite de caractères en mémoire, terminée par un octet nul.

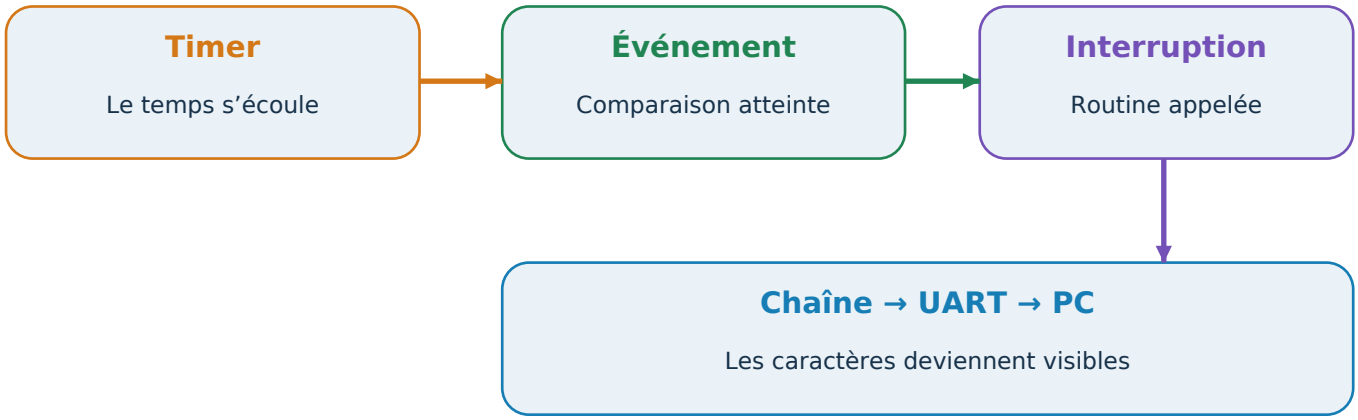
UART

Un périphérique qui transforme chaque octet en trame série.

TIMER

Un compteur matériel qui avance selon une horloge.

Le chemin d’un affichage périodique



La boucle vide n’empêche pas le matériel de travailler

Le CPU reste dans la boucle. Pendant ce temps, le timer continue de compter. Une interruption détourne brièvement l’exécution, puis le CPU revient dans la boucle. **Aucun test, aucun délai, aucun affichage dans cette boucle.**

Les pages suivantes expliquent les mécanismes ; les réglages de l’exercice restent à compléter.

Sources : datasheet, §6.7 p.15 ; §15.2 p.89-90 ; §19.2 p.143-144

Un caractère ≠ une chaîne

Les apostrophes, les guillemets et l’octet de fin n’ont pas le même rôle.

'A' : un caractère

Un seul caractère, représenté par une valeur numérique.

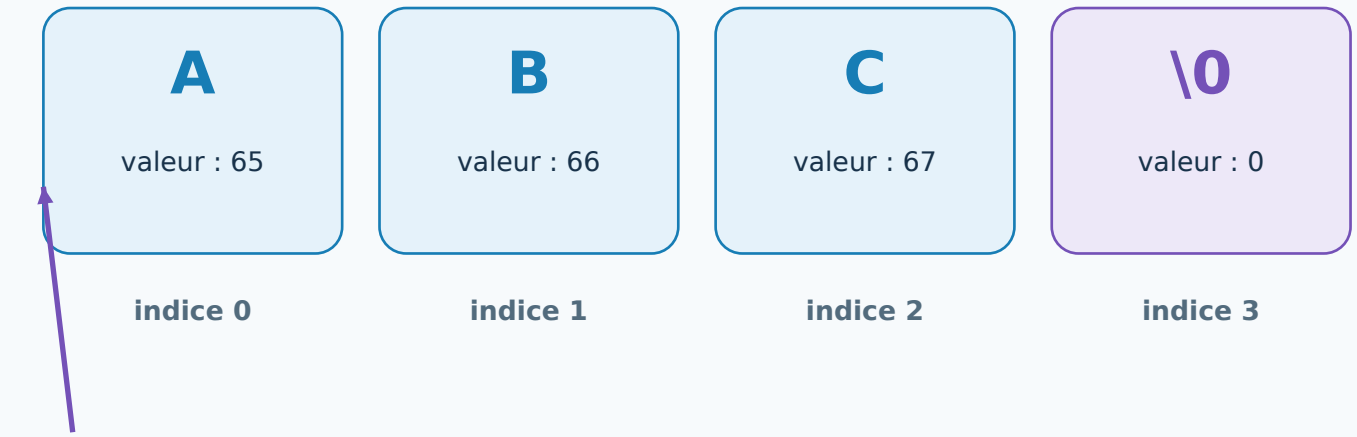
Il convient à une fonction qui reçoit un **char**.

"ABC" : une chaîne

Une suite de caractères, plus un **terminateur nul** ajouté par le langage C.

On transmet son adresse à une fonction.

Exemple distinct de l’exercice : **"ABC"** occupe **4 octets**. Les 3 lettres visibles sont suivies de **'\0'**.



Écriture	Signification
`'\0'`	Caractère nul : valeur 0, fin de la chaîne.
`'0'`	Chiffre zéro : valeur ASCII 48, caractère visible.
`'\n'`	Saut de ligne : caractère de contrôle, valeur ASCII 10.

Le terminateur ne fait pas partie du texte à afficher

Il sert à savoir où s’arrêter. L’envoyer au terminal n’est pas nécessaire.

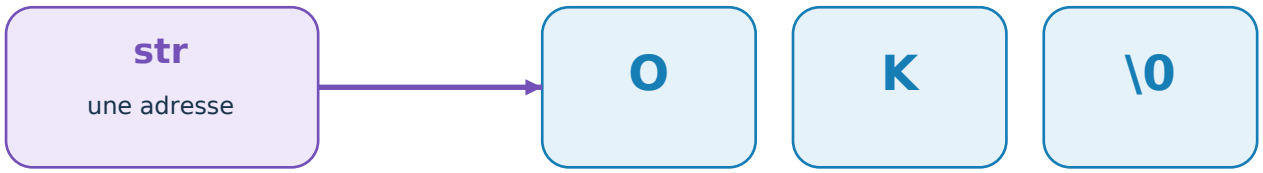
Comprendre const char* str

Un pointeur dit où lire ; const dit que les caractères ne doivent pas être modifiés.

```
void uart_printstr(const char* str);
```

Élément	Ce qu'il signifie
void	La fonction ne renvoie aucune valeur.
uart_printstr	Nom de la fonction qui transmet la chaîne.
const char*	Pointeur vers des caractères accessibles en lecture.
str	Nom du paramètre qui contient une adresse.
;	Déclaration : elle annonce la fonction ; son corps reste à écrire.

Adresse et contenu sont deux choses



Ce que const autorise

Tu peux **lire le caractère pointé** et **avancer le pointeur local**. Tu ne dois pas modifier le caractère à travers ce pointeur. Le pointeur passé en argument est copié : avancer str dans la fonction ne déplace pas le pointeur de l'appelant.

Du premier caractère au dernier

La fonction doit être générale ; le message demandé est seulement un argument.

Lire une case

*p : lit le caractère à l'adresse p.

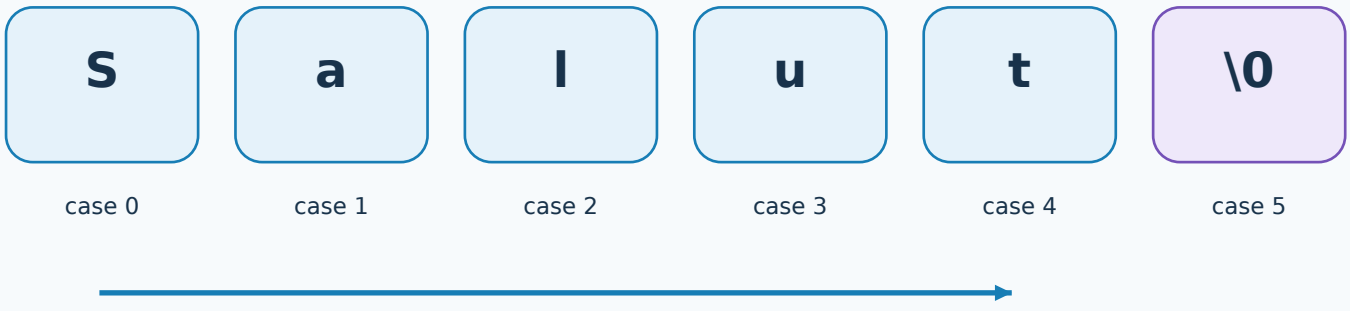
`str[i]` : lit le caractère d'indice i.

Changer de case

p++ : avance d'un caractère.

i++ : passe à l'indice suivant.

Tu peux choisir un parcours par **pointeur** ou par **indice**. Dans les deux cas, distingue le caractère à envoyer du terminateur qui arrête le parcours.



Le parcours avance, sans dépasser la case de fin.

Une ligne visible a aussi besoin de contrôles

\r ramène le curseur au début de la ligne. **\n** le fait descendre. La séquence **\r\n** est un choix courant pour un terminal série. Exemple différent : **"Salut\r\n"**. Le terminateur '\0' reste implicite.

Séparer la fonction et la présentation

Le prototype demande une fonction qui transmet la chaîne fournie. Décide où placer les caractères de fin de ligne : dans la chaîne ou dans l'appelant. Évite de fixer le texte ou sa longueur dans uart_printstr. Aucune fonction strlen ou printf n'est nécessaire.

L’USART transmet des octets

Le timer décide quand demander un message ; l’USART décide à quelle vitesse sortent les bits.

Registre / bit	Rôle à retenir
UBRR0H / UBRR0L	Diviseur qui règle le débit. 12 bits utiles au total.
U2X0 dans UCSR0A	Choix du facteur 16 ou 8 en mode asynchrone.
TXEN0 dans UCSR0B	Activation de l’émetteur sur PD1 / TXD.
UCSR0C et UCSZ02	Mode, parité, stop et taille des données.
UDRE0 dans UCSR0A	Tampon d’émission libre : un octet peut y être déposé.
UDR0	Registre de données : écrire y dépose l’octet à transmettre.

Deux étages, puis une broche



Le tampon libre n’est pas la fin de l’émission

UDRE0 indique qu’il reste une place pour un octet. **TXC0** indique que toute la trame a quitté le registre à décalage et que le tampon est vide. Un nouveau caractère peut donc être préparé pendant que des bits sortent encore.

8N1 : 1 bit de start + 8 bits de données + aucune parité + 1 bit de stop.

Sources : datasheet, §19.2 p.143-144 ; §19.4 p.147-148 ; §19.10 p.159-162

Le débit et le format restent à calculer

Le nouvel exercice conserve 115200 8N1, mais change le déclenchement des envois.

Mode normal

K = 16

UBRR idéal =
F_CPU / (K × débit) - 1

Double vitesse

K = 8

Débit réel =
F_CPU / [K × (UBRR + 1)]

UBRR est entier. Compare les entiers voisins et leurs débits réels. L’erreur relative est **100 × (débit réel / débit demandé - 1)**. Le choix dépend de la vraie fréquence, pas d’un nombre appris par cœur.

Choix	Champ à retrouver	Table / page
Asynchrone	UMSEL01:0	19-4 / p.161
Parité	UPM01:0	19-5 / p.161
Bits de stop	USBS0	19-6 / p.161
Bits de données	UCSZ02:0, sur B et C	19-7 / p.162

Masques : les outils que tu réutilises

`1 << k` sélectionne le bit k. `R & masque` le teste. `R |= masque` met les bits choisis à 1. `R &= ~masque` les met à 0. Vérifie les règles particulières de chaque registre avant une lecture-modification-écriture.

Même horloge réelle pour tes deux calculs

F_CPU doit correspondre à la fréquence effective du MCU. Une mauvaise valeur fausse **le débit UART et la période du timer**. Définir F_CPU ne configure pas l’horloge matérielle.

Sources : datasheet, tableau 19-1 p.146 ; tableaux 19-4 à 19-7 p.161-162 ; §19.11 p.163-165

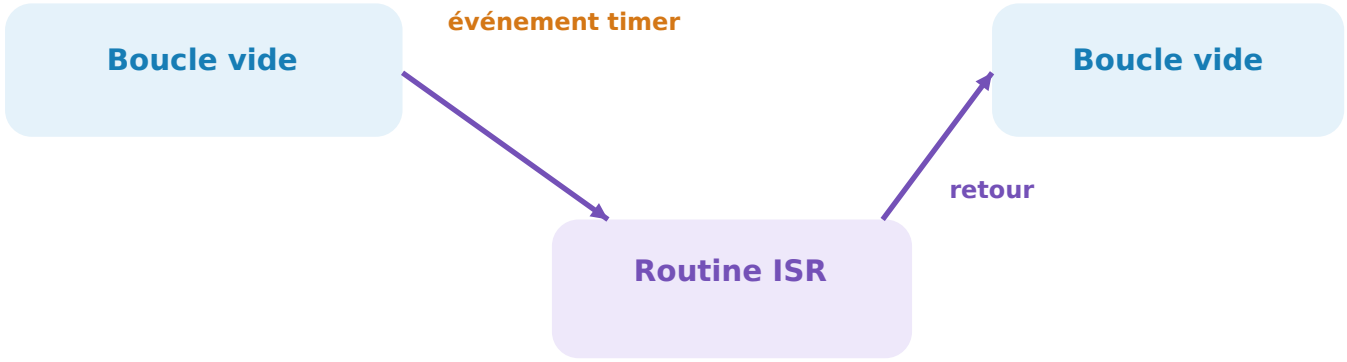
Une boucle vide, mais des événements

Une interruption est un détour temporaire de l’exécution du CPU.

Avant la boucle : préparer le fonctionnement

Le programme peut initialiser l’UART et le timer avant d’entrer dans la boucle. La contrainte porte sur **le corps de la boucle infinie** : il doit rester vide. L’attente de 2 secondes et la demande d’affichage se déclenchent via le matériel et sa routine d’interruption.

Exécution du CPU



Le timer continue de compter pendant ce détour.

Une ISR n’est pas appelée par ta boucle

ISR veut dire **Interrupt Service Routine**. Le CPU utilise le **vecteur d’interruption** lié à l’événement. La routine s’exécute, puis une instruction de retour d’interruption permet de reprendre le programme.

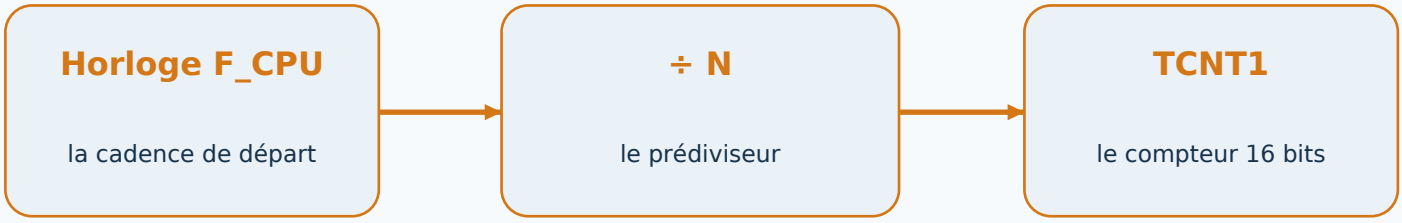
Les en-têtes autorisés

avr/io.h : noms des registres et bits du MCU choisi.
avr/interrupt.h : outils ISR, sei et cli.
util/delay.h : attentes actives autorisées, mais inutile pour mesurer ici une période dans une boucle qui doit rester vide.

Sources : datasheet, §6.3.1 p.11 ; §6.7 p.15-16 ; §11.1 p.49

Un timer mesure le temps en ticks

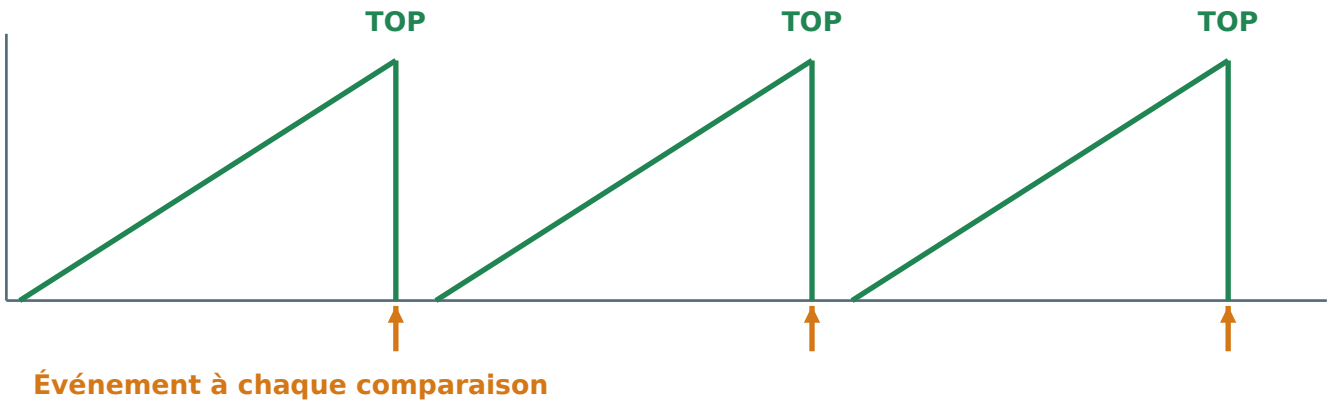
Le compteur avance dans le matériel, sans une instruction C à chaque incrément.



Le prédiviseur ralentit le compteur

Un tick dure **N / F_CPU** seconde. Pour Timer1, les divisions internes proposées sont **1, 8, 64, 256 et 1024**. Un compteur de 16 bits stocke des valeurs de **0 à 65535**.

CTC : compter, comparer, recommencer



CTC signifie Clear Timer on Compare Match. Avec **OCR1A comme TOP**, TCNT1 suit une séquence bornée par cette valeur. La comparaison produit un événement ; le compteur repart automatiquement. Une sortie OC1A n’est pas nécessaire pour demander une interruption.

Pourquoi Timer1 est un bon support pédagogique

Timer0 et Timer2 sont des compteurs 8 bits. Pour une longue période, ils demandent souvent de compter plusieurs événements en logiciel. Timer1 offre une plage plus grande. Ce guide développe la piste Timer1 en CTC ; d’autres approches restent possibles.

Sources : datasheet, §15.1-15.2 p.89-91 ; §15.9.2 et figure 15-6 p.100-101

Trouver le seuil de comparaison

Le +1 vient du fait que la séquence inclut la valeur 0.

$$T = N \times (OCR1A + 1) / F_CPU$$

$$OCR1A\ idéal = T \times F_CPU / N - 1$$

Pour chaque division N envisageable, calcule le nombre de ticks **C = T × F_CPU / N**. Une période directe tient dans Timer1 si **1 ≤ C ≤ 65536**, avec un seuil entier entre 0 et 65535. Recalcule ensuite la période réelle.

Exemple différent : 10 ms à 8 MHz

Avec N = 64 : tick = **8 µs**. Pour 10 ms, il faut **1250 ticks**, donc un seuil **1249**. Vérification : 64 × (1249 + 1) / 8 000 000 = **0,010 s**. Ce n’est pas le réglage de tes 2 secondes.

Entiers, arrondi et capacité

Un calcul entier tronque : évalue l’arrondi et sa conséquence sur T réel. Utilise des constantes assez larges, par exemple avec **UL**, pour éviter des produits en entier AVR 16 bits. Une valeur qui dépasse 65535 ne rentre pas dans OCR1A.

Le piège du facteur 2 dans la datasheet

La formule de p.101 avec **2 × N** concerne la fréquence d’une sortie OC1A qui **bascule** à chaque comparaison. Deux bascules font une période de signal. Pour la fréquence des **interruptions de comparaison**, il n’y a pas ce facteur 2.

Sources : datasheet, §15.9.2 p.100-101 ; tableau 15-6 p.110

Mode et horloge : TCCR1A / B

Les quatre bits WGM sont répartis dans deux registres.

TCCR1A

7	6	5	4	3	2	1	0
COM1A1	COM1A0	COM1B1	COM1B0	réservé	réservé	WGM11	WGM10

TCCR1B

7	6	5	4	3	2	1	0
ICNC1	ICES1	réservé	WGM13	WGM12	CS12	CS11	CS10

Champ / registre	Rôle	À consulter
WGM13:0	Choisit le mode et la source de TOP.	Tab.15-5, p.109
CS12:0	Arrêt, division de l'horloge ou horloge externe.	Tab.15-6, p.110
COM1A / COM1B	Action éventuelle sur les broches de comparaison.	Tab.15-2, p.108
TCNT1	Valeur courante du compteur.	§15.11.4, p.111
OCR1A	Valeur comparée au compteur ; TOP dans la piste choisie.	§15.11.5, p.111

Ce que tu dois retrouver dans les tables

Dans la table WGM, repère **CTC avec OCR1A comme TOP**, puis reporte les quatre bits dans les bons registres. Dans la table CS, retrouve l'encodage du N choisi. Laisse les sorties de comparaison déconnectées si tu veux uniquement un événement interne.

Sources : datasheet, §15.11.1-15.11.2 p.108-110 ; tableaux 15-2, 15-5 et 15-6

Un événement ne suffit pas à appeler l’ISR

L’indicateur, son autorisation locale et l’autorisation globale sont distincts.

TIMSK1 : autorisations d’interruption

7	6	5	4	3	2	1	0
réservé	réservé	ICIE1	réservé	réservé	OCIE1B	OCIE1A	TOIE1

TIFR1 : indicateurs d’événement

7	6	5	4	3	2	1	0
réservé	réservé	ICF1	réservé	réservé	OCF1B	OCF1A	TOV1

ÉVÉNEMENT → OCF1A mémorise une comparaison avec OCR1A.

LOCAL → OCIE1A autorise l’interruption de comparaison A.

GLOBAL → Le bit I de SREG autorise les interruptions du CPU.

sei et cli : les outils de l’en-tête interruptions

sei() active l’autorisation globale. **cli()** la coupe. Le bit global seul ne configure ni le timer ni l’autorisation locale. Relie toujours **la source d’événement, son masque et le bon vecteur**.

OCF1A : un indicateur effacé en écrivant 1

OCF1A s’efface automatiquement quand son vecteur d’interruption est exécuté. On peut aussi l’effacer en écrivant **1** à sa position. Pour effacer un événement ancien, utilise un masque ciblé ; une opération |= peut effacer d’autres indicateurs déjà à 1.

L’événement TIMER1 COMPA de la table p.49 correspond au nom AVR-LibC **TIMER1_COMPA_vect**. Ne le confonds pas avec le débordement TIMER1 OVF.

Sources : datasheet, §6.3.1 p.11 ; §15.11.8-15.11.9 p.112-113 ; §11.1 p.49

Faire court, puis revenir

L’ISR doit rester compatible avec la méthode d’émission UART choisie.

La macro ISR relie une routine à un vecteur

L’en-tête **avr/interrupt.h** fournit **ISR(VECTEUR)**. Dans une ISR standard, le compilateur prépare les sauvegardes nécessaires et le retour d’interruption. Tu écris le travail demandé par l’événement ; la boucle principale n’appelle pas cette routine.



Le timer fixe l’intervalle entre les demandes d’affichage.

Ne pas attendre 2 secondes dans la routine

La période appartient au timer. La routine répond à l’événement, puis rend la main. Avec une ISR standard, les autres interruptions sont temporairement masquées pendant son exécution.

Scrutation UART : pourquoi elle peut fonctionner ici

L’USART matériel continue d’émettre pendant une ISR. Une petite routine qui attend **UDRE0 par lecture directe** peut donc envoyer une courte chaîne. Sa durée doit rester faible devant 2 secondes ; elle retarde néanmoins les autres interruptions.

Éviter une attente qui dépend d’une autre ISR

Si l’envoi UART attend qu’une **autre interruption** vide une file logicielle, l’attente peut se bloquer dans une ISR standard. Vérifie donc la méthode de uart_tx. Pour cet exercice simple, une émission par scrutation n’exige pas d’interruptions UART.

Ta feuille de conception

Choisis les réglages avant d’assembler les fonctions du programme.

L’ordre logique d’initialisation

Prépare l’UART avant toute demande d’affichage. Configure le timer **à l’arrêt**, son mode, son seuil et son compteur. Traite un éventuel indicateur ancien. Autorise la source choisie et démarre le comptage dans un état cohérent ; ouvre les interruptions globales quand tout est prêt.

Point à décider	Ta réponse / tes calculs
Horloge effective / F_CPU	_____
Mode UART ; UBRR ; erreur	_____
Encodage 8N1 ; TX activé	_____
Timer ; mode ; source de TOP	_____
N ; nombre de ticks ; seuil	_____
Période réelle après arrondi	_____
WGM13:0 ; CS12:0	_____
Masque local ; bit global ; vecteur	_____
Parcours par indice ou pointeur	_____
Condition de fin ; fin de ligne	_____

Registres 16 bits sur un CPU 8 bits

Les noms **TCNT1** et **OCR1A** permettent un accès 16 bits en C ; le compilateur émet deux accès 8 bits. Le matériel exige haut puis bas à l’écriture. Initialise avant les ISR ; pour un accès partagé plus tard, il faut aussi protéger la séquence contre une interruption.

Sources : datasheet, §15.3 p.91-93 ; §15.11 p.108-113 ; §19.5 p.148

Lire les symptômes

Trois contraintes indépendantes : texte correct, cadence correcte et boucle vide.

Observation attendue

```
Hello World!
Hello World!
Hello World!
```

Des lignes distinctes, une demande d’affichage toutes les 2 secondes. À vérifier aussi dans le source : la fonction transmet son argument et le corps de la boucle infinie est vide.

Symptôme	Pistes de vérification
Rien ne s’affiche	Port et câblage ; UART prête ; timer en marche ; autorisations locale et globale ; vecteur correct.
Caractères illisibles	F_CPU réel ; UBRR et U2X0 ; format du terminal et du MCU.
Le texte continue avec du bruit	Terminaison '\0' ; parcours arrêté au bon endroit ; adresse de chaîne valide.
Tout reste sur une ligne	Présence des contrôles de fin de ligne ; comportement du terminal.
La période est doublée ou divisée	N ; seuil ; +1 ; confusion entre bascules OC1A et interruptions.
Le programme se bloque dans l’ISR	Attente UART qui dépend d’une autre ISR ; routine trop longue.

Ce qu’un indicateur ne mémorise pas

Un drapeau d’interruption est un bit, pas une file de tous les événements. Si plusieurs comparaisons surviennent pendant un long masquage, elles peuvent se regrouper en une seule demande en attente. D’où l’intérêt d’une routine courte.

Sources : datasheet, §15.9.2 p.100-101 ; §15.11.8 p.112 ; §19.10 p.159-162

Où trouver les informations

Références de ta datasheet ATmega328P, document 7810D-AVR-01/15

Notion recherchée	Section / tableau	Page(s)
Bit global d’interruptions	§6.3.1 : SREG	11
Entrée, retour, drapeaux, latence	§6.7 et §6.7.1	15-16
Vecteur Timer1 comparaison A	§11.1 ; tableau 11-1	49
Principe et capacité de Timer1	§15.1-15.2 ; figure 15-1	89-91
Accès aux registres 16 bits	§15.3	91-93
Comparaison et mode CTC	§15.7 ; §15.9.2	97-98 ; 100-101
Encodage WGM ; source de TOP	Tableau 15-5	109
Encodage des divisions CS	Tableau 15-6	110
TCNT1 ; OCR1A	§15.11.4-15.11.5	111
Masque local ; indicateur	§15.11.8-15.11.9	112-113
Broches TXD / RXD	§13.3.3	70-71
Débit, format et émission UART	§19.3-19.6 ; §19.10-19.11	145-150 ; 159-165

Les chaînes et les macros : hors de la datasheet

Les pointeurs, **const**, les chaînes et '\0' relèvent du **langage C**. La datasheet décrit le matériel ; elle ne définit pas ce prototype.

Les macros **ISR**, **sei**, **cli** et les noms se terminant par **_vect** sont documentés par AVR-LibC : Ouvrir la documentation officielle avr/interrupt.h

Schémas redessinés pour expliquer ; style repris du premier guide. Les exemples numériques et les chaînes d’illustration diffèrent de l’exercice.

Parcours conseillé : chaînes (p.2-4) → UART (p.5-6) → timer et interruptions (p.7-12) → feuille à compléter (p.13) → vérification (p.14).